

Bounding Byzantine Impact in Open CRDT Systems

Timothé Albouy
IMDEA Software Institute
Madrid, Spain

Carlos Baquero
Univ Porto & INESC TEC
Porto, Portugal

Abel Dantas
Univ Porto & CTVC
Porto, Portugal

Antonio Fernández Anta
IMDEA Software & Networks
Institutes
Madrid, Spain

Davide Frey
Inria, Univ Rennes, CNRS, IRISA
Rennes, France

Francisco Maia
Univ Porto & INESC TEC
Porto, Portugal

César Sánchez
IMDEA Software Institute
Madrid, Spain

Abstract

Conflict-free Replicated Data Types (CRDTs) enable available and eventually consistent data replication without coordination, making them well suited for open and partition-prone environments. Recent work has shown that CRDTs can be extended to tolerate Byzantine faults by ensuring that replicas eventually agree on the validity of operations, even in permissionless settings. However, validity alone does not prevent a Byzantine participant from inflicting unbounded damage by issuing large volumes of adversarial yet well-formed updates. For example, when editing text, an attacker can easily delete prior text. In this paper, we study how to bound the impact of Byzantine behavior in open CRDT systems. We introduce bounded Byzantine CRDTs, a rate-limiting framework for CRDTs in which each update carries an associated cost that limits the influence of adversarial operations relative to the resources they expend. Overall, this work bridges the gap between Byzantine-tolerant CRDTs and resource-bounded adversarial models, providing a principled foundation for deploying CRDTs in fully open, adversarial environments.

CCS Concepts: • Theory of computation → Distributed algorithms.

Keywords: CRDT, Byzantine fault tolerance, Permissionless systems, Proof of work.

ACM Reference Format:

Timothé Albouy, Carlos Baquero, Abel Dantas, Antonio Fernández Anta, Davide Frey, Francisco Maia, and César Sánchez. 2026. Bounding Byzantine Impact in Open CRDT Systems. In *13th International Workshop on Principles and Practice of Consistency for Distributed Data (PaPoC '26)*, April 27–30, 2026, Edinburgh, Scotland Uk. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3806077.3806698>

1 Introduction

Transient partitions in a distributed system can be seen as benign, non-Byzantine failures that affect availability but not safety, as messages are delayed rather than corrupted
DOI: 10.1145/3806077.3806698

or forged. Still, partitions have an impact on system design as they force a choice between preserving data consistency versus favouring availability [11]. Favouring availability implies accepting some level of data divergence. Conflict-free Replicated Data Types (CRDTs) [23] allow divergence and ensure that, when communication occurs, data eventually converges in a deterministic way. However, the standard model of CRDTs assumes a system model where replicas are correct and only subject to non-adversarial failures such as crashes, message loss, reordering, or network partitions.

More recently, a design for Byzantine fault-tolerant CRDTs [15] shows that CRDTs can tolerate Byzantine behavior even in open, permissionless environments, without requiring costly consensus algorithms. Based on cryptographic hashes of operations, such a design ensures that correct nodes eventually agree on whether an update is valid or not. In fact, under this model, any state update issued by a Byzantine node can only be accepted if it is indistinguishable from one coming from a correct node. However, even when operations are syntactically well-formed and accepted by correct nodes, a Byzantine attacker can still cause substantial harm.

A common use case for CRDTs is editable sequences, which enable shared text documents, as implemented in systems such as Automerge [12] and, as widely reported, in the iOS/macOS Notes app¹. In this setting, a Byzantine node may disrupt the system by overwriting or invalidating large portions of prior work. This behavior is common in Wikipedia edit wars (back-and-forth series of reverts between editors), and it is mitigated by a combination of social processes, moderation policies, and rate-limiting mechanisms rather than purely technical enforcement.

In cryptocurrency systems like Bitcoin, consensus ensures that no one can create coins out of thin air. However, an attacker can still transfer fractions of coins to multiple addresses, spamming recipients. While benign in Bitcoin due to BTC's value, such behavior constitutes genuine spam with

¹github.com/dunhamsteve/notesutils/blob/master/notes.md

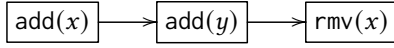


Figure 1. A totally ordered log representing a sequence of set operations, with final set state $\{y\}$ since x was removed.

speculative tokens, cluttering wallets with low- or zero-value assets and increasing storage requirements.

More generally, most permissionless systems are vulnerable to so-called Sybil attacks, where an attacker creates multiple identities to disrupt system operation. In these scenarios, such behavior would be considered correct by traditional Byzantine-tolerance algorithms. This calls for complementing Byzantine tolerance with rate-limiting mechanisms.

To this end, we propose a resource-bounded adversary model, orthogonal to Byzantine fault tolerance. We assume existing mechanisms ensure validity and convergence, and instead focus on bounding the rate and cumulative impact of valid operations. We propose a modified version of proof of work (PoW), in which the required computational effort is proportional to the impact of the associated CRDT operations, rather than to the raw operation count. This allows higher-impact operations to incur proportionally higher effort, thereby limiting the rate and consequences of adversarial behavior without sacrificing availability.

In this paper, we begin by describing the guarantees already achievable by systems that support CRDT liveness and convergence in Byzantine settings [15], and then analyze approaches to bounding Byzantine behavior in terms of the impact of individual operations. We offer examples of how our approach can be used and tailored to specific applications. We also discuss alternative strategies that could avoid PoW, e.g., using reputation or endorsement mechanisms.

2 Byzantine CRDTs

To understand how Byzantine CRDTs are constructed, we first review the properties of totally ordered logs and how they can be maintained in Byzantine settings. We then extend this discussion to partially ordered logs, which can accommodate partition tolerance.

2.1 Totally Ordered Logs

The value of a sequential data type is determined by its initial state and a totally ordered sequence of operations, see Figure 1. In a centralized system, this sequence can be represented by maintaining an append-only log. To prevent modifications to past operations, each log entry can include a cryptographic hash of the previous entry, thereby forming a hash-chained log. If the log is made public and replicated by multiple parties, any alteration of historical entries becomes detectable, as it would produce hashes different from those already observed and stored by other participants.

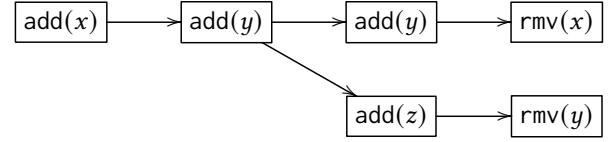


Figure 2. A partially ordered log where a total order up to $\text{add}(y)$ is followed by two concurrent sequences of operations. The final set state under add-wins semantics would be $\{y, z\}$, since $\text{add}(y)$ wins over concurrent $\text{rmv}(y)$.

Creating hash-chained logs and publishing them from a single location (namely, the operation issuer) ensures consistency, but not availability, as a single source of truth prevents conflicting versions even during network partitions. When multiple nodes are allowed to issue operations, two design options arise. (a) If the membership of issuing nodes is known, a consensus protocol [6, 17] can be used to agree on the next operation to append to the log. (b) If the system is open and potential issuers are unknown, Sybil-resistant consensus such as those using proof-of-work [8] can be used to probabilistically converge on the log that has been extended the furthest [19]. In the former case, network partitions primarily degrade liveness and availability, whereas in the latter, they primarily weaken finality.

A totally ordered log is a powerful construct that can support system-wide invariants such as the absence of double-spending, but it is inherently sensitive to network partitions. In contrast, highly available, partition-tolerant data types such as CRDTs can be expressed by relaxing the total-order requirement and considering partially ordered logs [10].

2.2 Partially Ordered Logs

On any node in a distributed system, the value of a CRDT can be determined by observing the partially ordered log (PO-log) known to that node [4]; see Figure 2. Nodes are free to append their own operations to the PO-log, propagate it to other nodes, and merge incoming PO-logs into their local PO-log. All these actions are partition-tolerant.

This construction can be made more robust by creating hash-chained PO-logs, a concept described as Merkle-DAGs in [22] and further developed in [15]. Updates are identified and chained in the PO-log by computing a cryptographic hash of each *update*, for example using SHA-256. An *update* includes the operation name, its parameters, and the lexicographically ordered list of predecessor hashes, if any.

However, in the above construction, unless node identifiers are included in the update, there is a potential for hash collisions when two nodes independently issue identical operations over identical logs. This ambiguity can be avoided by incorporating a replica identifier or a cryptographically strong nonce into the update, ensuring that each operation is identified even under concurrent, identical behavior.

In our design, we will assume that nodes possess public–private key pairs. The public key is considered to be the node’s ID. Hence, it is included in the update, and the hash of the update block is digitally signed using the corresponding private key. This establishes update authenticity, non-repudiation, and uniqueness. See an example in Figure 3.

We can further define the expected behavior of correct nodes in this setting.

- Correct nodes only extend and propagate PO-logs that are valid with respect to hash-chain integrity, digital signatures, and causal consistency.
- Correct nodes always include, as predecessors, the identities of the locally known maximal elements of the partial order when issuing new updates. This also ensures that each node sequentially extends its most recent operation.

Again, these procedures are partition tolerant. We now observe which Byzantine behaviors can (or cannot) be detected.

Detectable Byzantine Behavior. Given hash-chained PO-logs, digitally signed updates, and explicit causal dependencies, correct nodes can detect several classes of Byzantine behavior by verifying structural and cryptographic validity. Signatures allow detection of forgery and impersonation, as each update must be signed with the issuer’s private key. Hash chaining allows detection of history rewriting, causal inconsistencies, and malformed PO-logs, as each update references the hashes of its causal predecessors.

Undetectable Byzantine Behavior. Despite the above guarantees, some Byzantine behaviors remain undetectable, as they do not violate cryptographic or structural correctness. Byzantine nodes may issue operations that are syntactically valid, causally consistent, and correctly signed, yet semantically harmful (e.g., deleting or overwriting shared state or prior work). Such behavior is indistinguishable from legitimate CRDT operations.

The model also does not prevent or detect excessive operation issuance: a Byzantine node may generate an unbounded number of valid updates, leading to resource exhaustion or disproportionate impact. Nor does it prevent or detect Sybil behavior, where an adversary generates multiple identities to amplify its impact.

Thus, while cryptographic mechanisms detect invalid or inconsistent behavior, they do not bound the rate or impact of valid adversarial operations, motivating mechanisms that impose resource or access-control constraints.

3 Bounded Byzantine CRDTs

To motivate the need for bounding Byzantine behavior in CRDTs, we consider a simple counter CRDT that supports increment operations of the form `inc(n)`. In a benign setting,

correct nodes typically issue small increments reflecting local events or user actions (e.g., counting the number of detected events at multiple monitoring stations). However, in the presence of Byzantine participants, this assumption no longer holds. A Byzantine node may issue an update such as `inc(10 000)`, causing a disproportionate and immediate impact on the replicated state. While such behavior is syntactically valid and causally consistent, it violates the intended application semantics and overwhelms the contributions of correct nodes.

One possible mitigation is to restrict the maximum increment size, for example, by allowing only `inc(1)` operations. However, this approach is insufficient in open systems. An adversary may instead generate a large number of Sybil identities, each issuing a single `inc(1)` operation. The cumulative effect is equivalent to a single large increment, while remaining fully compliant with syntactic and semantic constraints.

This example illustrates a limitation of CRDTs in open, adversarial environments: bounding the impact of individual operations is insufficient without also bounding the rate or total impact of participating replicas. In the next section, we show how introducing a computational cost per operation can address this limitation by constraining the aggregate impact of Byzantine participants.

3.1 Proof-of-Work

A proof-of-work mechanism can be attached to CRDT instances operating in open, adversarial environments to limit the rate and cumulative impact of valid operations.

We consider two functions: a *cost function* $C(\cdot)$ and a *work function* $W(\cdot)$. The cost function

$$C(op, pk, polog) = s$$

deterministically computes a target work amount s for a given operation op and identity pk , optionally taking into account the local PO-log $polog$. This function is data-type dependent and encodes application-specific notions of proportional contribution.

The work function $W(d \cdot s, h)$ produces a proof that an amount of computational effort proportional to s has been expended under a difficulty parameter d , where

$$h = H(op, pk, pred)$$

is the hash of the update to be appended, and

$$pred = \langle H_1, \dots, H_k \rangle_{lex}$$

is the lexicographically ordered list of predecessor hashes. The difficulty parameter d is locally computed, converges system-wide, and regulates the overall cost of work, $cost = d \cdot s$; for now, let us consider $d = 1$; we will discuss adaptive difficulty adjustment in Section 3.1.3.

Concretely, W may be instantiated by searching for a nonce such that the hash of the nonce appended to h satisfies

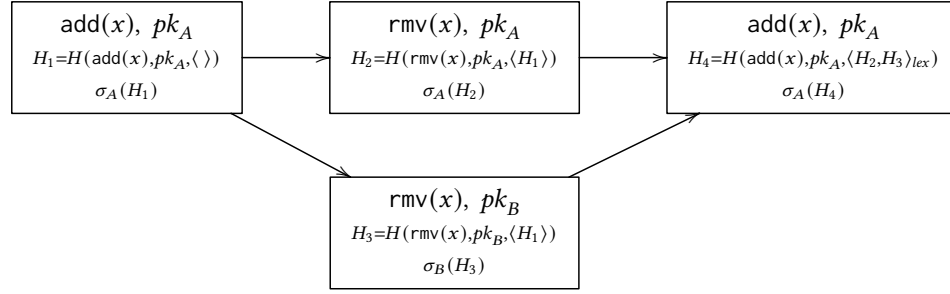


Figure 3. A simple hash-chained partially ordered log. Each update block includes the operation, the issuer public key (ID), the lexicographically ordered list of predecessor hashes, the hash over the previous fields, and a digital signature of that hash.

a difficulty predicate (e.g., a required number of leading zeros), similar to standard hash-based proof-of-work schemes. Verification of the resulting proof is efficient, while generation incurs a computational cost proportional to $d \cdot s$.

We now illustrate this approach with two examples.

Counters. As discussed previously, in a counter CRDT supporting operation $\text{inc}(n)$, a Byzantine node pk may attempt to issue a single update with a large increment. Under the proposed scheme, the cost function enforces $C(\text{inc}(n), pk, \text{polog}) = n$, requiring work proportional to the increment size. This mechanism also mitigates Sybil attacks.² An adversary controlling many identities and issuing k operations of $\text{inc}(1)$ must perform the same total amount of work as issuing a single $\text{inc}(k)$, regardless of how the increments are distributed across replicas.

Editable Sequences. For editable sequence CRDTs, the cost function may be defined as proportional to the size of the update. For example, $\text{insert}(\text{index}, \text{text})$ may incur a cost proportional to the length of text , while $\text{delete}(\text{index}, \text{size})$ incurs a cost proportional to size .

More elaborate policies are also possible. By inspecting the local polog , the cost function may increase the required work for edits targeting regions that have experienced a high volume of prior modifications. This way, zones of the document that receive more attention progressively become harder to change, while preserving availability and convergence.

We provide additional instantiations of the cost function sets and registers in our companion technical report [1].

3.1.1 Resource-Bounded Adversary Model. We model Byzantine participants as resource-bounded adversaries whose ability to impact the system is constrained by the amount of computational effort they can expend. Each update issued by a node must be accompanied by a valid PoW whose cost is determined by the target function $C(op, pk, \text{polog})$ and the difficulty parameter d .

²In these examples, we assume a single datatype and CRDT instance. Systems supporting multiple datatypes in a shared PO-log are not considered for now and should be subject to further analysis.

An adversary is characterized by a finite computational budget B , representing the total amount of work it can perform over some period of time. This budget may be distributed arbitrarily across identities (taking into account Sybil attacks). Regardless of its strategy, the adversary can inject into the system updates whose total number and cumulative impact are bounded by B .

Correct nodes are assumed to verify proofs of work and accept only updates whose proofs satisfy the required difficulty. Verification is efficient and does not depend on the adversary's budget.

3.1.2 Bounded Impact of Byzantine Operations. We say that a CRDT instance satisfies the *bounded impact* property if, for any execution and any adversary with computational budget B , the cumulative effect of all updates issued by the adversary is bounded by a function of B , independent of the number of identities under its control. Formally, let U_A be the set of updates issued by an adversary A , and let

$$\sum_{u \in U_A} C(u) \leq B.$$

Then the total semantic impact of adversarial updates on the replicated state is proportional to B . In particular, distributing work across many replicas does not increase the adversary's impact beyond what its computational budget permits.

This property directly addresses the limitations identified in earlier sections. In the absence of proof-of-work enforcement, an adversary may issue an unbounded number of valid operations or amplify its impact through Sybil identities. Under the proposed scheme, such behavior remains syntactically valid but quantitatively constrained.

Importantly, bounded impact does not prevent Byzantine nodes from issuing semantically harmful operations; rather, it limits the rate and cumulative magnitude of such harm. Correct nodes keep progressing independently, preserving the availability and convergence guarantees of CRDTs.

3.1.3 Adaptive Difficulty and Fairness. The difficulty parameter d regulates the expected computational cost of issuing updates and plays a central role in balancing fairness

and throughput. While a fixed difficulty may be sufficient in static environments, in open systems, the available computational power and update demand may vary significantly over time. To accommodate such variability, the difficulty parameter can be adjusted dynamically in a decentralized and partition-tolerant manner.

Since a PO-log is a partial order rather than a sequence, we avoid reasoning about prefixes and instead consider a replica's *causally closed view* V of the PO-log. A view V contains a set of updates such that, for every update in V , all of its causal predecessors are also in V . Difficulty adjustment is derived solely from information recorded in such views, without relying on synchronized clocks or coordination.

Each accepted update u carries an associated target cost $s_u = C(u)$, which is either included explicitly or deterministically re-computable from the operation and its causal context. To measure progress in the PO-log without relying on wall-clock time, we define the *work time* of a causally closed view V as

$$\text{WT}(V) = \sum_{u \in V} C(u) = \sum_{u \in V} s_u,$$

which provides a deterministic and monotonic notion of progress that is preserved under merges.

To define difficulty-adjustment epochs in a partial order, we associate each update u with an intrinsic work-time coordinate derived from its causal past. Let $\text{Past}(u)$ denote the set of causal predecessors of u . We define $t(u) = \text{WT}(\text{Past}(u))$ and assign u to the epoch

$$\text{epoch}(u) = \left\lfloor \frac{t(u)}{E} \right\rfloor,$$

for a fixed epoch size E . This assignment is independent of any total order and is preserved under merges, allowing all replicas to deterministically classify updates into epochs based solely on the PO-log structure. Figure 4 illustrates epoch assignment in a sample PO-log.

To adapt difficulty as computational resources and participation evolve, and to maintain a relatively stable temporal epoch duration, each update may additionally include and sign a locally measured time delta δ_u , representing the elapsed real time spent on the issuer's PoW. These values are not individually trusted; instead, replicas aggregate them across all updates assigned to the same epoch using a robust statistic (e.g., the median), with Byzantine manipulation bounded as discussed in Section 5. Let

$$\widehat{\Delta}_e = \text{median}\{\delta_u \mid u \in V \wedge \text{epoch}(u) = e\}$$

be the estimated duration of epoch e in a replica's view V .

Given a target epoch PoW duration Δ^* , e.g. 200ms, replicas compute the next difficulty as

$$d_{e+1} = d_e \cdot \frac{\Delta^*}{\widehat{\Delta}_e}.$$

If updates are produced too quickly relative to the target, difficulty increases; if progress is slow, difficulty decreases. This mechanism mirrors the spirit of difficulty adjustment in proof-of-work blockchains. Unlike totally ordered proof-of-work blockchains, in which mining nodes engage in a winner-takes-it-all proof-of-work competition, our proposal is much lighter. Only the node issuing an operation has to do the work imposed by the cost function, and thus, there is no wasted work from competing nodes. In addition, the value chosen by the cost function may be low enough to make it acceptable to humans.³

Temporary divergence in difficulty estimates may occur during network partitions, as replicas observe different causally closed views. However, since difficulty is a deterministic function of the PO-log contents, replicas converge on the same difficulty once communication resumes and views are merged. No consensus or leader election is required.

Adaptive difficulty contributes to fairness by ensuring that participants with comparable computational resources achieve comparable long-term impact, independent of the number of identities they control. Importantly, tuning d affects only the rate at which updates can be issued, not their validity or semantics. Correct nodes remain free to issue operations at any time, subject only to the computational effort required. As a result, adaptive difficulty preserves availability and convergence while preventing sustained high-rate attacks that could otherwise overwhelm the system.

While the mechanisms described here rely on simple epoch-based heuristics, more sophisticated policies are possible. For example, difficulty adjustment may incorporate alternative aggregators and sliding windows, or even reputation systems based on historical behavior. We explore such behavior-based alternatives in Section 6.

4 Related Work

Recent work has extended CRDTs to Byzantine environments through different approaches. Jacob *et al.* [13, 14] show that state-based CRDTs are inherently equivocation-tolerant. Almeida and Shapiro [3] introduce the blocklace, a DAG-based CRDT that eventually detects and excludes Byzantine equivocators through signed hash chains. Kleppmann and Howard [16] define Byzantine Eventual Consistency for applications inherently immune to Sybil attacks. Sanjuán *et al.* [22] formalize Merkle-CRDTs, which use hash-chained Merkle-DAG structures to provide causal ordering and efficient syncing in decentralized peer-to-peer settings. While their work lays the cryptographic foundation for content-addressed CRDT operations, it explicitly assumes non-adversarial replicas and does not address Sybil resistance or rate-limiting of Byzantine operations. Cholvi *et al.* [7] present BFT Distributed Grow-only Sets (BDSO), achieving consensus-free eventual consistency

³Rate limiting is also common to limit password bruteforcing attacks.

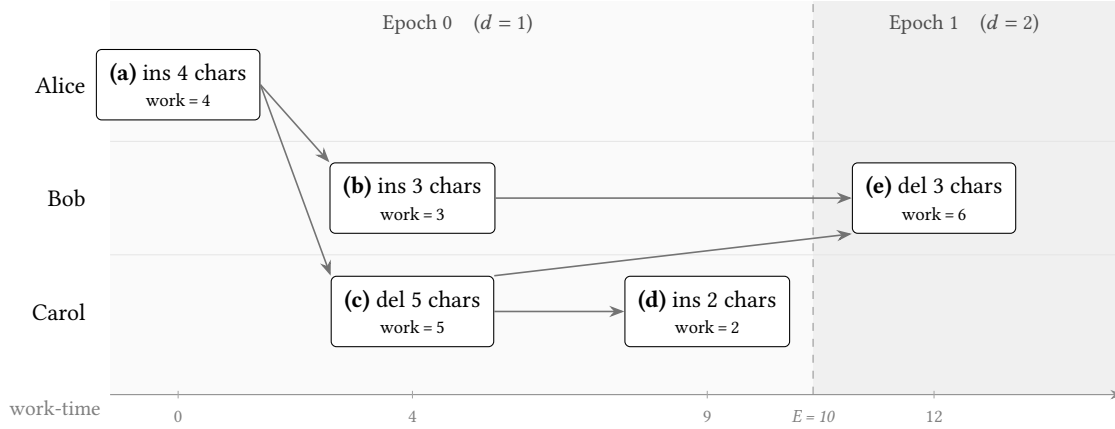


Figure 4. Epoch assignment in a PO-log with epoch size $E = 10$. Updates are positioned by the cumulative cost of their causal past. Concurrent updates (d) and (e) fall into different epochs because (e) merges both branches ($t = 12$) while (d) sees only one ($t = 9$). Update (e) deletes 3 characters but incurs work = 6 because difficulty doubles in epoch 1 ($d = 2$).

through Byzantine Reliable Broadcast with quorum-based validation.

Consensus-free Byzantine tolerance has also been explored through relaxed ordering models. Frey *et al.* [10] introduce Process-Commutative distributed objects that relax total ordering requirements to FIFO order per process, achieving Strong Eventual Consistency in Byzantine settings using Mazurkiewicz traces. Their work demonstrates that Byzantine transactional systems can operate without consensus when operations exhibit appropriate commutativity properties, providing a theoretical foundation for understanding the minimal ordering requirements of distributed objects.

Proof-of-work, introduced by Dwork and Naor [8] and popularized by Bitcoin [19], provides Sybil resistance by tying impact to computational resources. While primarily used in consensus protocols, PoW's rate-limiting properties have not been applied to bound Byzantine impact in coordination-free CRDTs. Our work bridges this gap by using PoW to *proactively limit* the update rate of any node, Byzantine or otherwise, thereby bounding adversarial impact *proportionally to computational resources* rather than detecting misbehavior after the fact or relying on membership assumptions.

Finally, the idea of using endorsements or reputation to manage access to CRDT objects is related to prior work on CRDTs and access control [2, 21]. For instance, Renaux *et al.* [21] propose SRDTs (Secure Replicated Data Types), which associate access rights with CRDT operations to enforce security policies at data replicas. Meanwhile, Almeida [2] discusses identity-management challenges in the context of large-scale CRDT-based systems.

Other work has explored how CRDTs can be used to implement access-control primitives. For example, Weber and Bieniusa [24] show that causally consistent replication can achieve convergence of access-control policies without relying on undo operations, while Yanakieva *et al.* [25] study

CRDT-based access-control in the context of distributed storage. Rault *et al.* [20] propose mechanisms to combine access control with collaborative editing over CRDT-replicated data. Finally, Frey *et al.* [9] show that certain access-control objects, such as deny-lists, require consensus for synchronization, albeit only among a limited set of participants. Nonetheless, to the best of our knowledge, using endorsements and reputation as Sybil-resilience mechanisms in the context of CRDT-based replication remains unexplored.

5 Security Discussion

Our adaptive difficulty mechanism may incorporate per-update elapsed-time reports δ_u supplied by issuers. Since Byzantine participants can choose δ_u arbitrarily, a natural concern is whether adversaries can bias difficulty updates by manipulating these values.

Threat. A Byzantine issuer may report extreme values (e.g., very small δ_u to suggest that epochs complete too quickly, or very large δ_u to suggest sluggish progress) in an attempt to drive difficulty d up or down. If successful, this could either reduce liveness (by inflating d) or enable cheap spam bursts (by deflating d).

Mitigations. We propose two design choices that limit the effectiveness of such manipulations.

First, δ_u values are not interpreted individually. Difficulty updates aggregate δ_u over an epoch using a robust statistic such as the median (or a trimmed mean). This makes the update rule resilient to a bounded number of outliers: a minority of adversarial reports cannot arbitrarily skew $\hat{\Delta}_e$.

Second, PoW itself constrains the adversary's ability to flood epochs with timestamp reports. Injecting many misleading δ_u values requires issuing many updates, incurring proportional work. Thus, even if an adversary attempts to

gain impact over $\hat{\Delta}_e$ by increasing its sample share, it must pay the corresponding computational cost.

Overall, adversarial timestamp manipulation can affect the responsiveness of adaptive difficulty but does not compromise safety, convergence, or the bounded-impact guarantees: operations remain subject to proof-of-work verification, and any attempt to dominate the difficulty signal requires proportional computational expenditure. Future work could explore verifiable timing mechanisms such as PoSW (Proof of Sequential Work) [18] or VDFs (Verifiable Delay Functions) [5] to strengthen difficulty adaptation.

Threat. A Byzantine node may issue causally valid operations anchored in the past to benefit from lower difficulty.

Mitigations. In the CRDT model, such operations are indistinguishable from those issued by partitioned honest nodes. As a result, preventing backdated low-cost injections requires restricting honest nodes as well. One mitigation direction is to impose an integration cost for branches where the causally closed view is proportionally much smaller than a more complete view to integrate upon.

6 Limitations and Alternatives to PoW

While PoW provides a mechanism to bound the cumulative impact of Byzantine operations, it does not eliminate semantic attacks entirely. A sufficiently resourced adversary (such as a state actor or well-funded organization) can still mount harmful operations by expending proportional computational effort. The bounded impact property ensures such attacks are quantitatively constrained, but the cost affects different Byzantine agents heterogeneously: well-resourced attackers face lower barriers than individual malicious nodes.

Moreover, proof-of-work imposes computational and energy costs on all participants, including honest nodes. However, unlike competitive PoW systems such as Bitcoin, where many nodes compete to find a single winning hash and all other computational work is wasted, our approach utilizes every proof: each operation’s computational cost contributes directly to the replicated state. The energy footprint is therefore orders of magnitude lower than blockchain mining. Nevertheless, even this reduced cost may be seen as undesirable, motivating exploration of alternative mechanisms.

An alternative direction leverages reputation systems based on historical behavior. Since operations in Byzantine CRDTs are cryptographically signed, and we have the history of operations within the PO-log, it is possible to attribute beneficial contributions to specific nodes. Consider two application scenarios. In a distributed sensing network monitoring environmental conditions, nodes consistently reporting values within statistically expected bounds could accumulate reputation, which they could later spend on higher-impact operations such as flagging aberrant readings or disputing

reports from other nodes. Similarly, in a collaborative editing system, nodes whose edits are not reverted over the last several epochs could earn reputation spendable on reverting others’ edits or making larger modifications. Such reputation-based permission systems could reduce PoW requirements for established cooperative participants and constitute barriers for new and potentially untrusted nodes. Another, somewhat symmetrical approach could rely on nodes endorsing one another, leveraging cryptographically signed certificates.

Another set of beneficial contributions is protocol-level CRDT operations. Merge operations, where a node creates an update referencing multiple concurrent maximals in the PO-log, directly contribute to system convergence by reducing DAG branching and carry no semantic ambiguity. These operations are (1) *verifiable* from the signed PO-log structure without trusting node claims, (2) *application-agnostic* and thus do not require domain-specific definitions of beneficial behavior. Nodes that consistently perform merge operations could accumulate reputation redeemable for reduced proof-of-work requirements on application-level operations.

Still, designing reputation or endorsement mechanisms for open, partition-tolerant environments introduces three fundamental challenges. First, a larger system necessarily presents a larger attack surface: Byzantine nodes can strategically farm reputation, or endorse one another, through superficially cooperative behavior before exploiting accumulated trust for malicious ends. Second, defining “beneficial behavior” is inherently application-domain specific and might be harder to narrow down formally. Third, embedding permission schemes into CRDT data structures requires new abstractions that would need to encode evolving memberships and reputation scores, potentially exploding complexity on both the state representation and merge semantics.

Hybrid approaches combining both proof-of-work and proof-of-behavior might be the most practical path forward. For example, in a large-scale distributed sensing network where nodes dynamically join and leave, proof-of-work could provide Sybil resistance at the network edge, requiring computational work from new or unknown participants, while established nodes with positive behavioral history operate under reduced work requirements.

7 Acknowledgments

Carlos Baquero is partially funded by national funds through *Fundação para a Ciência e a Tecnologia* (FCT), I.P., under project UID/50014/2025. Abel Dantas acknowledges support from *Cooperativa Tecnológica de Viana do Castelo* (CTVC). Davide Frey acknowledges support from the Alvearium partnership between Inria and Hivenet. This work is partially funded by MICIU/AEI/10.13039/501100011033, ERDF, and the ESF+ under grant DRONAC (PID2022-140560OB-I00); and by MICIU/AEI/10.13039/501100011033 under grant CEX2024-001471-M.

References

- [1] Timoth   Albouy, Carlos Baquero, Abel Dantas, Antonio Fern  ndez Anta, Davide Frey, and Francisco Maia. Bounding Byzantine impact in open CRDT systems. <https://doi.org/10.5281/zenodo.19368001>, 2026.
- [2] Paulo S  rgio Almeida. Approaches to conflict-free replicated data types. *ACM Comput. Surv.*, 57(2), November 2024.
- [3] Paulo S  rgio Almeida and Ehud Shapiro. The blocklace: A Byzantine-repelling and universal conflict-free replicated data type. *arXiv preprint arXiv:2402.08068*, 2024.
- [4] Carlos Baquero, Paulo S  rgio Almeida, and Ali Shoker. Making operation-based CRDTs operation-based. In *Proceedings of the First Workshop on Principles and Practice of Eventual Consistency*, PaPEC '14, New York, NY, USA, 2014. Association for Computing Machinery.
- [5] Dan Boneh, Joseph Bonneau, Benedikt B  nz, and Ben Fisch. Verifiable delay functions. In *Advances in Cryptology – CRYPTO 2018*, volume 10991 of *Lecture Notes in Computer Science*, pages 757–788. Springer, 2018.
- [6] Miguel Castro and Barbara Liskov. Practical Byzantine fault tolerance. In *Proceedings of the Third Symposium on Operating Systems Design and Implementation (OSDI)*, pages 173–186, 1999.
- [7] Vicent Cholvi, Antonio Fern  ndez Anta, Chryssis Georgiou, Nicolas Nicolaou, Michel Raynal, and Antonio Russo. Byzantine-tolerant distributed grow-only sets: Specification and applications. *arXiv preprint arXiv:2103.08936*, 2021.
- [8] Cynthia Dwork and Moni Naor. Pricing via processing or combatting junk mail. In *Advances in Cryptology – CRYPTO '92*, pages 139–147, 1992.
- [9] Davide Frey, Mathieu G  stin, and Michel Raynal. The synchronization power (consensus number) of access-control objects: the case of allowlist and denylist. In Rotem Oshman, editor, *37th International Symposium on Distributed Computing (DISC 2023)*, volume 281 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 21:1–21:23, Dagstuhl, Germany, 2023. Schloss Dagstuhl – Leibniz-Zentrum f  r Informatik.
- [10] Davide Frey, Lucie Guillou, Michel Raynal, and Fran  ois Ta  ani. Process-commutative distributed objects: From cryptocurrencies to Byzantine-fault-tolerant CRDTs. *arXiv preprint arXiv:2311.13936*, 2023.
- [11] Seth Gilbert and Nancy Lynch. Brewer’s conjecture and the feasibility of consistent, available, partition-tolerant web services. *SIGACT News*, 33(2):51–59, 2002.
- [12] Alex Good, Joseph Gentle, and Martin Kleppmann. Automerger. <https://github.com/automerger/automerger>, 2017.
- [13] Florian Jacob, Saskia Bayreuther, and Hannes Hartenstein. On conflict-free replicated data types and equivocation in Byzantine setups. In *Distributed Computing Systems (DISC 2021 Posters)*, 2021. *arXiv:2109.10554*.
- [14] Florian Jacob, Saskia Bayreuther, and Hannes Hartenstein. On CRDTs in Byzantine environments. In Christian Wressnegger, Delphine Reinhardt, Thomas Barber, Bernhard C. Witt, Daniel Arp, and Zolt  n   d  m Mann, editors, *Sicherheit, Schutz und Zuverl  ssigkeit: Konferenzband der 11. Jahrestagung des Fachbereichs Sicherheit der Gesellschaft f  r Informatik e.V. (GI), Sicherheit 2022, Karlsruhe, Germany, April 5–8, 2022*, LNI, pages 113–126. Gesellschaft f  r Informatik e.V., 2022.
- [15] Martin Kleppmann. Making CRDTs Byzantine fault tolerant. In *Proceedings of the 9th Workshop on Principles and Practice of Consistency for Distributed Data (PaPoC@EuroSys 2022)*, pages 8–15. ACM, 2022.
- [16] Martin Kleppmann and Heidi Howard. Byzantine eventual consistency and the fundamental limits of peer-to-peer databases. *arXiv preprint arXiv:2012.00472*, 2020.
- [17] Leslie Lamport. The part-time parliament. *ACM Transactions on Computer Systems*, 16(2):133–169, 1998.
- [18] Mohammad Mahmoodi, Tal Moran, and Salil Vadhan. Publicly verifiable proofs of sequential work. In *Proceedings of the 4th Conference on Innovations in Theoretical Computer Science (ITCS 2013)*, pages 373–388. ACM, 2013.
- [19] Satoshi Nakamoto. Bitcoin: A peer-to-peer electronic cash system. <https://bitcoin.org/bitcoin.pdf>, 2008.
- [20] Pierre-Antoine Rault, Claudia-Lavinia Ignat, and Olivier Perrin. Distributed access control for collaborative applications using CRDTs. In *Proceedings of the 9th Workshop on Principles and Practice of Consistency for Distributed Data*, PaPoC '22, page 33–38, New York, NY, USA, 2022. Association for Computing Machinery.
- [21] Thierry Renaux, Sam Van den Vonder, and Wolfgang De Meuter. Secure RDTs: Enforcing access control policies for offline available JSON data. *Proc. ACM Program. Lang.*, 7(OOPSLA2), October 2023.
- [22] Hector Sanjuan, Samuli Poyhtari, Pedro Teixeira, and Ioannis Psaras. Merkle-CRDTs: Merkle-DAGs meet CRDTs, 2020.
- [23] Marc Shapiro, Nuno Pregui  a, Carlos Baquero, and Marek Zawirski. Conflict-free replicated data types. *Stabilization, Safety, and Security of Distributed Systems*, 6976:386–400, 2011.
- [24] Mathias Weber, Annette Bieniusa, and Arnd Poetzsch-Heffter. Access control for weakly consistent replicated information systems. In Gilles Barthe, Evangelos Markatos, and Pierangela Samarati, editors, *Security and Trust Management*, pages 82–97, Cham, 2016. Springer International Publishing.
- [25] Elena Yanakieva, Michael Youssef, Ahmad Hussein Rezae, and Annette Bieniusa. Access control conflict resolution in distributed file systems using CRDTs. In *Proceedings of the 8th Workshop on Principles and Practice of Consistency for Distributed Data*, PaPoC '21, New York, NY, USA, 2021. Association for Computing Machinery.